

Decision trees

Dataset: Forest CoverType, seven species

How to use these notes

The primary text for this lecture is Géron, Chapter 5. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

Contents

1	A model family chosen by the shape of the answer	2
1.1	The brief	2
1.2	The data, and what we did to it	3
2	The mathematics: impurity, and the objective CART minimises	3
2.1	The objective CART minimises	4
2.2	Where the two criteria can disagree	4
2.3	Measure it rather than accepting it	4
3	Growing a tree, and reading one	4
3.1	Fit one with no constraints	4
3.2	What depth actually buys	5
3.3	Overruling the grid	5
3.4	Reading the tree	6
3.5	Reading one prediction	6
4	One number for seven species is a summary	7

5	The notebook	8
6	Exercises	8
	Summary	8

1 A model family chosen by the shape of the answer

Every model so far predicts with a weighted sum $\theta^\top \mathbf{x}$, fitted by solving a linear system or descending a convex surface. Such models need their features scaled, because the geometry of the loss depends on the units; they reach curves only by manufacturing the curve yourself; and their explanation of a prediction is a sum of n terms — a reason, but not a short one.

Today’s model is not a weighted sum at all. It is a sequence of yes/no questions, and *the sequence is the explanation*.

1.1 The brief

An agency managing a national forest must publish a map of forest cover type — which species dominates each 30 by 30 metre patch — over an area far too large to survey on foot. From cartographic measurements alone (elevation, slope, aspect, distances to water, road and fire ignition points, hillshade, soil type) predict which of seven species covers the patch.

The map goes into the public record. It decides where logging is permitted, which parcels qualify for habitat protection, and how fire risk is modelled for the district. A landowner refused a permit is entitled to ask why. So is a court.

The requirement that decides the model

Every *individual* prediction must be accompanied by a human-readable justification: a statement, in terms of the measured quantities, of why this patch was classified as that species.

Note the word *individual*. Not “which variables matter in general”. Not “the model is 88% accurate”. This patch, this answer, this reason.

“Human-readable” is not a testable condition, so it has to be negotiated into one before any code is written.

Requirement	Testable form
Readable by a non-specialist	stated as conditions on the measured columns, not on transformed ones
Short enough to check	at most 8 conditions per prediction
Auditable	the same patch gives the same justification, and the rule can be applied by hand

Eight is the number the agency chose. It is arbitrary — and it is now a hyperparameter fixed by the brief rather than by cross-validation. That is unusual and worth noticing.

1.2 The data, and what we did to it

581,012 patches, each with 54 measurements: 10 quantitative, 4 already one-hot for wilderness area, 40 already one-hot for soil type. No missing values, no text columns, nothing to impute. This one arrives clean, so the difficulty is entirely elsewhere.

We use 60,000 patches, stratified so the class proportions are preserved exactly, split 48,000 / 12,000. Stratifying on the label matters here: with one class at 0.5% of the data, an unstratified split can hand a fold almost none of it.

Failure condition — numbers measured on a subsample

The subsample is a compromise for speed. Every number in this lecture is measured on it, and every one of them would move on the full set.

Subsampling for speed is a legitimate choice and a *reportable* one. Subsampling and not mentioning it is not.

Two species are 85% of the ground. A classifier that always says “Lodgepole Pine” scores **48.8%**, so anything below that is worse than a constant and the distance above it is the only part you earned.

2 The mathematics: impurity, and the objective CART minimises

You called `DecisionTreeClassifier().fit()`. It chose one feature and one threshold at every node. Chose them to minimise *what*?

A node holds m instances over K classes, with p_k the fraction in class k . The class distribution $\mathbf{p}$ is the *only* thing an impurity measure sees. A node is **pure** when some $p_k = 1$.

Gini impurity is

$$G(\mathbf{p}) = \sum_k p_k(1 - p_k) = 1 - \sum_k p_k^2.$$

Draw two of the node’s instances independently with replacement; $\sum_k p_k^2$ is the probability they are the same class, so G is the probability they *differ*. A pure node never disagrees with itself.

Entropy is

$$H(\mathbf{p}) = - \sum_k p_k \log_2 p_k,$$

with $0 \log_2 0 = 0$ by the limit: the average number of bits needed to transmit the class of an instance drawn from the node, under the best possible code. A pure node needs none.

2.1 The objective CART minimises

A question $x_k \leq t_k$ sends m_L of the node's m instances left and m_R right. Its cost is the impurity I — Gini or entropy — of the two children, weighted by their size:

$$J(k, t_k) = \frac{m_L}{m} I_L + \frac{m_R}{m} I_R,$$

searched over every feature k and every threshold t_k . Equivalently, maximise $\Delta I = I_{\text{parent}} - J$; the parent term is the same for every candidate at this node, so the two are the same search.

2.2 Where the two criteria can disagree

Both are zero at a pure node and grow as the classes mix, so they usually rank candidate splits the same way. How far can they part? On a two-class node, G has maximum $1/2$ and H has maximum 1 , so compare $H/2$ with G : both are 0 at $p \in \{0, 1\}$ and $1/2$ at $p = 1/2$, and the largest gap anywhere else is **0.055**, at $p \approx 0.10$ and again at $p \approx 0.90$. Two candidates whose weighted impurities differ by more than 0.055 are ranked identically by both criteria. Disagreement needs a close call.

2.3 Measure it rather than accepting it

Does the choice matter? That is a question about behaviour, and we have a dataset. Twenty 80% resamples of the training set, both criteria fitted at depth 8 on the same resample each time, so the comparison is paired and the resample-to-resample noise cancels. Gini is above entropy on 17 of the 20, and the paired difference, entropy minus Gini, is -0.44 ± 0.39 points.

Real, and not worth tuning

The effect is *real*: Gini wins on 17 of the 20 resamples. The effect is *tiny*: 0.44 points, against the 8.0 points that `max_depth` is worth later in this lecture. So do not spend your tuning budget here; leave the default.

3 Growing a tree, and reading one

Each internal node asks $x_k \leq t_k$ about one feature; each leaf predicts the majority class of the training instances that reached it. Predicting one instance costs $O(\log_2 m)$ — one root-to-leaf path — and does not depend on n , which is why reading a prediction costs the same as making it. That is what makes the justification free.

Trees need no feature scaling at all: $x_k \leq t_k$ is invariant under any strictly increasing transformation of x_k , so standardising a column changes its thresholds and nothing else.

3.1 Fit one with no constraints

Depth 33, 5,699 leaves, 100.0% training accuracy. With 48,000 training patches that is eight patches per leaf on average, so of course it is perfect on its own data.

Unconstrained tree	Accuracy
On the training set	100.0%
5-fold cross-validated, on the training set	82.1%
On the test set (once, at the end)	82.6%

82% is real and far above the 48.8% baseline, so the tree family works here. But count what a prediction rests on: the tree is 33 levels deep, its mean justification is **17.88** conditions, and the brief allows eight.

Two hyperparameters take that depth away. `max_depth` is a hard ceiling on the length of every path — and therefore on the justification — and `min_samples_leaf` rejects any split that would leave a child with fewer training instances than its value.

3.2 What depth actually buys

Between depth 8 and depth 18 there are **8.0 points** of accuracy. That is the measured price of the eight-condition requirement, and it is a number worth putting in front of the agency.

Failure condition — the one hyperparameter we may not tune

Cross-validation says `max_depth=None`. The brief says `max_depth=8`. Cross-validation does not get a vote, because it is not optimising the thing the agency is buying.

This is the first time in the course that the honest answer to “what does the search say?” is “the search is answering the wrong question”. It will not be the last. Bring the 8.0-point figure to the agency; perhaps they will trade a condition or two for it. That is a conversation, not a `GridSearchCV`.

We search the whole grid anyway — including the depths we may not use — because the rows we cannot pick are exactly what tells the agency what its requirement costs.

3.3 Overruling the grid

The grid’s answer under the cap is `min_samples_leaf=1`. We ship 20 anyway.

Why

The model states its justification as “91% of the 463 training patches in this leaf”. At `min_samples_leaf=1` that becomes “100% of the 1” — a single surveyed patch wearing the grammar of evidence. The brief requires a justification that can be audited; that is not one. Cross-validation prefers 1 at 73.75%; the brief requires 20 at 73.35%. The price of auditability is 0.40 points, and it goes to the agency. A constraint that overrules a measurement has to show the measurement it overruled.

	Unconstrained	Depth 8, leaf 20
Leaves	5,699	163
Columns consulted, of 54	47	24
Mean conditions per justification	17.88	7.97
Smallest leaf	1	20
Training accuracy	100.0%	74.4%
Cross-validated accuracy	82.1%	73.4%

3.4 Reading the tree

`export_text` needs no plotting library at all, and is what you paste into an email. The first three questions the shipped model asks about any patch are about elevation (the root, at 3,046 metres), and then elevation again or wilderness area and soil type.

Each drawn node carries five fields. The question (Elevation <= 3046.5, go left if true); gini, the G of this node, 0 for a single species; `samples`, the training patches that reached it; `value`, their class distribution **p**; and `class`, the majority — what a leaf here would predict.

3.5 Reading one prediction

The entire justification mechanism is `tree_.children_left`, `tree_.feature` and `tree_.threshold` — nine lines, no library beyond what is already imported. For one test patch:

```
1. Elevation = 3,763 > 3,046
2. Elevation = 3,763 > 3,306
3. Wild_2    = 1 > 0
4. Soil_31   = 0 <= 0
5. Elevation = 3,763 > 3,360
6. HDist_Fire = 3,020 > 364
7. Shade_Noon = 200 <= 244
8. Shade_3pm = 133 <= 196
-> Krummholz (91% of the 463 training patches in this leaf)
```

Eight conditions, every one a measurement a surveyor took, and the true species of that patch is Krummholz. The path *is* the justification: no extra machinery, no approximation, no second model explaining the first.

A note on thresholds: CART puts the split midway between two adjacent observed values, which is why the root reads 3046.5 rather than 3046. Nothing in the data sits at 3,046.5 metres.

Failure condition — read the leaf carefully

91% of the 463 training patches in that leaf were Krummholz. That is a statement about 463 *training patches*, not a probability that this patch is Krummholz.

`predict_proba` returns exactly those leaf proportions, so a tree’s “probabilities” are piecewise constant and identical for every patch reaching the same leaf. Across the 163 leaves the smallest holds 20 patches and the largest several thousand; those deserve very different amounts of trust, and the count belongs in the justification.

The honest sentence is “of the training patches that satisfied these eight conditions, 91% were Krummholz” — not “this patch is 91% likely to be Krummholz”.

4 One number for seven species is a summary

The test set, once: **73.0%** on 12,000 patches, against a constant baseline of 48.8%, with every choice made on cross-validated folds of the training set.

Cover type	Share of the data	Recall
Lodgepole Pine	48.8%	82.0%
Ponderosa Pine	6.2%	80.8%
Spruce/Fir	36.5%	69.6%
Krummholz	3.5%	56.4%
Cottonwood/Willow	0.5%	56.1%
Douglas-fir	3.0%	12.8%
Aspen	1.6%	2.6%

The row-normalised confusion matrix (`normalize="true"`, so each row reads “of the patches that really were this species, where did they go?”) sends 94% of the Aspen row to Lodgepole Pine. The model finds fewer than 3 Aspen patches in every hundred. In the overall accuracy that costs 1.6 points and is invisible.

Failure condition — why the tree cannot see Aspen

J weights each child’s impurity by *how many instances it holds*. A split that isolates a class at 1.6% of the node moves the weighted average by almost nothing, so the greedy search never chooses one.

It is a property of the objective, not a bug in the implementation — read the definition of J again and you could have predicted it.

`class_weight="balanced"` multiplies each instance’s contribution by the inverse of its class frequency. Aspen recall rises to most of the class, because the split that isolates it is now worth choosing; overall accuracy falls sharply, because the two species that are 85% of the ground now carry 2/7 of the weight instead of 85% of it. A different model answering a different question — and quoting either number without the other is an argument rather than a measurement.

5 The notebook

What to change

Raise `max_depth` from 8 to 12 and re-run the trace. The depth sweep says what that buys in accuracy; the justification gets longer — up to twelve conditions where the brief allows eight — and that pair is the trade this whole lecture is about.

6 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 7's deck and in the exercise book.

1. A node holds 100 training patches: 50 of one species, 30 of a second and 20 of a third. Compute its Gini impurity, and give the value a pure node would have. [3]
2. On a two-class problem, entropy halved and Gini never differ by more than 0.055 at any node. Two candidate splits of the same node have weighted Gini impurities 0.30 and 0.38. Can entropy rank them the other way round? Answer, with a reason. [4]
3. An unconstrained tree with 5,699 leaves scores 100.0% on its 48,000 training patches. Explain why that number says nothing about new patches, and name the number to report instead. [4]
4. Under the depth-8 cap the grid prefers `min_samples_leaf=1` (73.75% cross-validated) to 20 (73.35%). Give the reason to ship 20 anyway, and say what must be reported with it. [4]
5. Per-class recall shows one class far below the rest. Give two responses that do not involve changing the model, and say which you would try first. [4]

Summary

The model family was chosen by the shape of the required answer rather than by the data: a tree hands a court an individual justification in the measured columns, because its root-to-leaf path *is* the justification. Gini impurity is the probability that two draws from a node differ, entropy the bits needed to transmit a class; CART chooses each split to minimise their size-weighted average over the two children. On a two-class node the two criteria part by at most 0.055, so they disagree only on close calls, and the paired measurement agreed: 0.44 points between them against the 8.0 that depth is worth.

The brief fixed `max_depth=8`, which cost 8.0 points, and auditability fixed `min_samples_leaf=20`, which cost 0.40 more. Both figures go to the agency, because a constraint that overrules a measurement has to show the measurement it overruled. The shipped tree scores 73.0% against a 48.8% constant, with a mean justification of 7.97 conditions — and 2.6% recall on Aspen, because *J* weights each child by its size. Reweighting the classes raises that recall and lowers the accuracy: a different model answering a different question.